

SQL Practical Examination

Complete Answer Guide

All Problem Statements | Create Tables | Queries | Expected Outputs

PS 1 & 2 — Student Course Management System

Step 1: Create Tables & Insert Data (Run Once)

```
-- RUN THIS FIRST BEFORE ANY QUERIES

CREATE TABLE Instructor (
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(50),
    Department VARCHAR(50)
);

CREATE TABLE Course (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(50),
    Credits INT,
    InstructorID INT,
    FOREIGN KEY (InstructorID) REFERENCES Instructor(InstructorID)
);

CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Email VARCHAR(50),
    Age INT,
    Address VARCHAR(100)
);

CREATE TABLE Enrollment (
    EnrollmentID INT PRIMARY KEY,
    StudentID INT,
    CourseID INT,
    EnrollmentDate DATE,
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID),
    FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
);

INSERT INTO Instructor VALUES
(1, 'Dr. Sharma', 'sharma@college.com', 'AI&DS'),
(2, 'Dr. Patil', 'patil@college.com', 'CS'),
(3, 'Dr. Mehta', 'mehta@college.com', 'IT'),
(4, 'Dr. Joshi', 'joshi@college.com', 'Mech'),
(5, 'Dr. Singh', 'singh@college.com', 'Civil');

INSERT INTO Course VALUES
(1, 'AI&DS', 4, 1), (2, 'DBMS', 3, 2), (3, 'OS', 3, 3), (4, 'CN', 3, 4), (5, 'Python', 2, 5);

INSERT INTO Student VALUES
(1, 'Aarav Joshi', 'aarav@mail.com', 20, 'Pune'),
(2, 'Priya Sharma', 'priya@mail.com', 21, 'Mumbai'),
(3, 'Ravi Patil', 'ravi@mail.com', 19, 'Pune'),
(4, 'Sneha More', 'sneha@mail.com', 22, 'Nashik'),
(5, 'Kiran Shah', 'kiran@mail.com', 18, 'Pune');

INSERT INTO Enrollment VALUES
(1, 1, 1, '2024-06-01'), (2, 2, 2, '2024-06-01'),
(3, 3, 1, '2024-06-02'), (4, 4, 3, '2024-06-03'), (5, 5, 1, '2024-06-04');
```

PS 1 — Queries

Q1. Show Students from AI&DS Course

```
SELECT s.StudentID, s.Name, s.Email
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID
WHERE c.CourseName = 'AI&DS';
```

Output:

StudentID	Name	Email
1	Aarav Joshi	aarav@mail.com
3	Ravi Patil	ravi@mail.com
5	Kiran Shah	kiran@mail.com

Q2. Find Students Living in Pune

```
SELECT * FROM Student WHERE Address LIKE '%Pune%';
```

Output:

StudentID	Name	Email	Age	Address
1	Aarav Joshi	aarav@mail.com	20	Pune
3	Ravi Patil	ravi@mail.com	19	Pune
5	Kiran Shah	kiran@mail.com	18	Pune

Q3. Sort Students by Name Alphabetically

```
SELECT * FROM Student ORDER BY Name ASC;
```

Output:

StudentID	Name	Email	Age	Address
1	Aarav Joshi	aarav@mail.com	20	Pune
5	Kiran Shah	kiran@mail.com	18	Pune
2	Priya Sharma	priya@mail.com	21	Mumbai
3	Ravi Patil	ravi@mail.com	19	Pune
4	Sneha More	sneha@mail.com	22	Nashik

Q4. Count Total Students

```
SELECT COUNT(*) AS TotalStudents FROM Student;
```

Output:

TotalStudents
5

Q5. Find Average Student Age

```
SELECT AVG(Age) AS AverageAge FROM Student;
```

Output:

AverageAge
20.0000

PS 2 — Queries

Q1. Find Students Between Age 18 and 22

```
SELECT * FROM Student WHERE Age BETWEEN 18 AND 22;
```

Output:

StudentID	Name	Email	Age	Address
1	Aarav Joshi	aarav@mail.com	20	Pune
2	Priya Sharma	priya@mail.com	21	Mumbai
3	Ravi Patil	ravi@mail.com	19	Pune
4	Sneha More	sneha@mail.com	22	Nashik
5	Kiran Shah	kiran@mail.com	18	Pune

Q2. Display Student Names with Course Names

```
SELECT s.Name AS StudentName, c.CourseName
FROM Student s
JOIN Enrollment e ON s.StudentID = e.StudentID
JOIN Course c ON e.CourseID = c.CourseID;
```

Output:

StudentName	CourseName
Aarav Joshi	AI&DS
Priya Sharma	DBMS
Ravi Patil	AI&DS
Sneha More	OS
Kiran Shah	AI&DS

Q3. Display Instructor Names with Courses

```
SELECT i.Name AS InstructorName, c.CourseName
FROM Instructor i
JOIN Course c ON i.InstructorID = c.InstructorID;
```

Output:

InstructorName	CourseName
Dr. Sharma	AI&DS
Dr. Patil	DBMS
Dr. Mehta	OS
Dr. Joshi	CN
Dr. Singh	Python

Q4. Count Courses Handled by Each Instructor

```
SELECT i.Name, COUNT(c.CourseID) AS TotalCourses
FROM Instructor i
JOIN Course c ON i.InstructorID = c.InstructorID
GROUP BY i.Name;
```

Output:

Name	TotalCourses
Dr. Sharma	1
Dr. Patil	1
Dr. Mehta	1
Dr. Joshi	1
Dr. Singh	1

Q5. Maximum Course Credits

```
SELECT MAX(Credits) AS MaxCredits FROM Course;
```

Output:

MaxCredits
4

PS 3 & 4 — Employee Management System

Step 1: Create Tables & Insert Data (Run Once)

```
CREATE TABLE Department (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50),  
    Location VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    EmployeeID INT PRIMARY KEY,  
    EmployeeName VARCHAR(50),  
    Email VARCHAR(50),  
    Salary DECIMAL(10,2),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);  
  
CREATE TABLE Project (  
    ProjectID INT PRIMARY KEY,  
    ProjectName VARCHAR(50),  
    Budget DECIMAL(12,2),  
    DepartmentID INT,  
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)  
);  
  
CREATE TABLE Payroll (  
    PayrollID INT PRIMARY KEY,  
    EmployeeID INT,  
    Bonus DECIMAL(10,2),  
    Deduction DECIMAL(10,2),  
    Salary DECIMAL(10,2),  
    FOREIGN KEY (EmployeeID) REFERENCES Employee(EmployeeID)  
);  
  
INSERT INTO Department VALUES  
(1, 'IT', 'Pune'), (2, 'Finance', 'Mumbai'), (3, 'AI&DS', 'Nashik'),  
(4, 'HR', 'Pune'), (5, 'Marketing', 'Mumbai');  
  
INSERT INTO Employee VALUES  
(1, 'Amit Sharma', 'amit@mail.com', 55000, 1),  
(2, 'Priya Singh', 'priya@mail.com', 62000, 2),  
(3, 'Ravi Kumar', 'ravi@mail.com', 71000, 3),  
(4, 'Sneha Patil', 'sneha@mail.com', 48000, 4),  
(5, 'Karan More', 'karan@mail.com', 83000, 1);  
  
INSERT INTO Project VALUES  
(1, 'AI Research', 500000, 3), (2, 'Finance App', 300000, 2),  
(3, 'HR Portal', 150000, 4), (4, 'ML Pipeline', 800000, 3), (5, 'ERP System', 600000, 1);  
  
INSERT INTO Payroll VALUES  
(1, 1, 5000, 2000, 55000), (2, 2, 8000, 3000, 62000),  
(3, 3, 10000, 4000, 71000), (4, 4, 3000, 1500, 48000), (5, 5, 12000, 5000, 83000);
```

PS 3 — Queries

Q1. Retrieve Employees from IT Department

```
SELECT e.EmployeeID, e.EmployeeName, e.Email, e.Salary  
FROM Employee e  
JOIN Department d ON e.DepartmentID = d.DepartmentID
```

```
WHERE d.DepartmentName = 'IT';
```

Output:

EmployeeID	EmployeeName	Email	Salary
1	Amit Sharma	amit@mail.com	55000.00
5	Karan More	karan@mail.com	83000.00

Q2. Sort Employees by Salary in Descending Order

```
SELECT * FROM Employee ORDER BY Salary DESC;
```

Output:

EmployeeID	EmployeeName	Email	Salary	DeptID
5	Karan More	karan@mail.com	83000.00	1
3	Ravi Kumar	ravi@mail.com	71000.00	3
2	Priya Singh	priya@mail.com	62000.00	2
1	Amit Sharma	amit@mail.com	55000.00	1
4	Sneha Patil	sneha@mail.com	48000.00	4

Q3. Find Average Salary

```
SELECT AVG(Salary) AS AverageSalary FROM Employee;
```

Output:

AverageSalary
63800.000000

Q4. Count Total Employees

```
SELECT COUNT(*) AS TotalEmployees FROM Employee;
```

Output:

TotalEmployees
5

Q5. Sort Projects by Budget

```
SELECT * FROM Project ORDER BY Budget ASC;
```

Output:

ProjectID	ProjectName	Budget	DeptID
3	HR Portal	150000.00	4
2	Finance App	300000.00	2
1	AI Research	500000.00	3
5	ERP System	600000.00	1
4	ML Pipeline	800000.00	3

PS 4 — Queries (Joins + Index)

Q1. Display Employee with Department Name = AI&DS

```
SELECT e.EmployeeName, e.Email, d.DepartmentName, d.Location
FROM Employee e
JOIN Department d ON e.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'AI&DS';
```

Output:

EmployeeName	Email	DepartmentName	Location
Ravi Kumar	ravi@mail.com	AI&DS	Nashik

Q2. Display Projects with Department = AI&DS

```
SELECT p.ProjectName, p.Budget, d.DepartmentName
FROM Project p
JOIN Department d ON p.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'AI&DS';
```

Output:

ProjectName	Budget	DepartmentName
AI Research	500000.00	AI&DS
ML Pipeline	800000.00	AI&DS

Q3. Display Employee Payroll Information

```
SELECT e.EmployeeName, p.Bonus, p.Deduction, p.Salary
FROM Employee e
JOIN Payroll p ON e.EmployeeID = p.EmployeeID;
```

Output:

EmployeeName	Bonus	Deduction	Salary
Amit Sharma	5000.00	2000.00	55000.00
Priya Singh	8000.00	3000.00	62000.00
Ravi Kumar	10000.00	4000.00	71000.00
Sneha Patil	3000.00	1500.00	48000.00
Karan More	12000.00	5000.00	83000.00

Q4. Employees Working in Finance Department

```
SELECT e.EmployeeName, e.Email, e.Salary
FROM Employee e
JOIN Department d ON e.DepartmentID = d.DepartmentID
WHERE d.DepartmentName = 'Finance';
```

Output:

EmployeeName	Email	Salary
Priya Singh	priya@mail.com	62000.00

Q5. Create Index on Employee Name

```
CREATE INDEX idx_emp_name ON Employee(EmployeeName);  
-- Output: Query OK, 0 rows affected
```

Output:

Result
Query OK, 0 rows affected (index created successfully)

PS 5 & 6 — Hostel Management System

Step 1: Create Tables & Insert Data (Run Once)

```
CREATE TABLE Room (  
    RoomID INT PRIMARY KEY,  
    RoomNumber VARCHAR(10),  
    RoomType VARCHAR(20),  
    Capacity INT,  
    Floor INT  
);  
  
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    StudentName VARCHAR(50),  
    Gender VARCHAR(10),  
    Course VARCHAR(50),  
    RoomID INT,  
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID)  
);  
  
CREATE TABLE Staff (  
    StaffID INT PRIMARY KEY,  
    StaffName VARCHAR(50),  
    Position VARCHAR(50),  
    Salary DECIMAL(10,2)  
);  
  
CREATE TABLE Fee (  
    FeeID INT PRIMARY KEY,  
    StudentID INT,  
    TotalFee DECIMAL(10,2),  
    PaidAmout DECIMAL(10,2),  
    DueAmout DECIMAL(10,2),  
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)  
);  
  
INSERT INTO Room VALUES  
(1,'R101','Single',1,1), (2,'R102','Double',2,1),  
(3,'R201','Double',2,2), (4,'R202','Triple',3,2), (5,'R301','Single',1,3);  
  
INSERT INTO Student VALUES  
(1,'Aarav Joshi','Male','AI&DS',1),  
(2,'Priya Sharma','Female','CS',2),  
(3,'Ravi Patil','Male','IT',2),  
(4,'Sneha More','Female','Mech',4),  
(5,'Kiran Shah','Male','Civil',5);  
  
INSERT INTO Staff VALUES  
(1,'Mr. Desai','Warden',45000), (2,'Mrs. Kulkarni','Clerk',30000),  
(3,'Mr. Nair','Security',25000), (4,'Mrs. Joshi','Cook',22000),  
(5,'Mr. Rao','Maintenance',20000);  
  
INSERT INTO Fee VALUES  
(1,1,50000,50000,0), (2,2,50000,30000,20000),  
(3,3,50000,50000,0), (4,4,50000,10000,40000), (5,5,50000,40000,10000);
```

PS 5 — Queries

Q1. Display Student Names and Courses

```
SELECT StudentName, Course FROM Student;
```

Output:

StudentName	Course
Aarav Joshi	AI&DS
Priya Sharma	CS
Ravi Patil	IT
Sneha More	Mech
Kiran Shah	Civil

Q2. Retrieve Students Staying in Room 2

```
SELECT * FROM Student WHERE RoomID = 2;
```

Output:

StudentID	StudentName	Gender	Course	RoomID
2	Priya Sharma	Female	CS	2
3	Ravi Patil	Male	IT	2

Q3. Sort Staff by Salary Descending

```
SELECT * FROM Staff ORDER BY Salary DESC;
```

Output:

StaffID	StaffName	Position	Salary
1	Mr. Desai	Warden	45000.00
2	Mrs. Kulkarni	Clerk	30000.00
3	Mr. Nair	Security	25000.00
4	Mrs. Joshi	Cook	22000.00
5	Mr. Rao	Maintenance	20000.00

Q4. Count Total Students

```
SELECT COUNT(*) AS TotalStudents FROM Student;
```

Output:

TotalStudents
5

Q5. Calculate Total Due Amount

```
SELECT SUM(DueAmount) AS TotalDue FROM Fee;
```

Output:

TotalDue
70000.00

PS 6 — Queries (Joins + Index)

Q1. Display Student with Room Details

```
SELECT s.StudentName, s.Course, r.RoomNumber, r.RoomType, r.Floor
FROM Student s
JOIN Room r ON s.RoomID = r.RoomID;
```

Output:

StudentName	Course	RoomNumber	RoomType	Floor
Aarav Joshi	AI&DS	R101	Single	1
Priya Sharma	CS	R102	Double	1
Ravi Patil	IT	R102	Double	1
Sneha More	Mech	R202	Triple	2
Kiran Shah	Civil	R301	Single	3

Q2. Display Student Fee Details

```
SELECT s.StudentName, f.TotalFee, f.PaidAmout, f.DueAmout
FROM Student s
JOIN Fee f ON s.StudentID = f.StudentID;
```

Output:

StudentName	TotalFee	PaidAmout	DueAmout
Aarav Joshi	50000.00	50000.00	0.00
Priya Sharma	50000.00	30000.00	20000.00
Ravi Patil	50000.00	50000.00	0.00
Sneha More	50000.00	10000.00	40000.00
Kiran Shah	50000.00	40000.00	10000.00

Q3. Student Paying Highest Fee

```
SELECT s.StudentName, f.TotalFee
FROM Student s
JOIN Fee f ON s.StudentID = f.StudentID
ORDER BY f.TotalFee DESC LIMIT 1;
```

Output:

StudentName	TotalFee
Aarav Joshi	50000.00

Q4. Students Staying in Double Rooms

```
SELECT s.StudentName, r.RoomType
FROM Student s
JOIN Room r ON s.RoomID = r.RoomID
WHERE r.RoomType = 'Double';
```

Output:

StudentName	RoomType
Priya Sharma	Double

StudentName	RoomType
Ravi Patil	Double

Q5. Create Index on Room Number

```
CREATE INDEX idx_room_number ON Room(RoomNumber);
```

Output:

Result
Query OK, 0 rows affected (index created successfully)

PS 11 & 12 — Hospital Management System

Step 1: Create Tables & Insert Data (Run Once)

```
CREATE TABLE Patient (  
    PatientID INT PRIMARY KEY,  
    PatientName VARCHAR(50),  
    Gender VARCHAR(10),  
    Age INT,  
    Disease VARCHAR(50)  
);  
  
CREATE TABLE Doctor (  
    DoctorId INT PRIMARY KEY,  
    DoctorName VARCHAR(50),  
    Specialization VARCHAR(50),  
    Salary DECIMAL(10,2),  
    Experience INT  
);  
  
CREATE TABLE Appointment (  
    AppointmentID INT PRIMARY KEY,  
    PatientID INT,  
    DoctorId INT,  
    AppointmentDate DATE,  
    RoomNo INT,  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID),  
    FOREIGN KEY (DoctorId) REFERENCES Doctor(DoctorId)  
);  
  
CREATE TABLE Billing (  
    BillID INT PRIMARY KEY,  
    PatientID INT,  
    TotalAmount DECIMAL(10,2),  
    PaidAmount DECIMAL(10,2),  
    DueAmount DECIMAL(10,2),  
    FOREIGN KEY (PatientID) REFERENCES Patient(PatientID)  
);  
  
INSERT INTO Patient VALUES  
(1,'Aarav Joshi','Male',45,'Diabetes'),  
(2,'Priya Sharma','Female',35,'Fever'),  
(3,'Ravi Patil','Male',52,'BP'),  
(4,'Sneha More','Female',28,'Asthma'),  
(5,'Kiran Shah','Male',60,'Heart');  
  
INSERT INTO Doctor VALUES  
(1,'Dr. Mehta','Cardiologist',80000,10),  
(2,'Dr. Patil','General',55000,5),  
(3,'Dr. Sharma','Neurologist',90000,15),  
(4,'Dr. Joshi','Orthopedic',70000,8),  
(5,'Dr. Singh','Dermatologist',65000,7);  
  
INSERT INTO Appointment VALUES  
(1,1,1,'2024-07-01',101),(2,2,2,'2024-07-02',102),  
(3,3,3,'2024-07-03',103),(4,4,4,'2024-07-04',104),(5,5,5,'2024-07-05',105);  
  
INSERT INTO Billing VALUES  
(1,1,15000,15000,0),(2,2,5000,3000,2000),  
(3,3,20000,20000,0),(4,4,8000,5000,3000),(5,5,25000,10000,15000);
```

PS 11 — Queries

Q1. Retrieve Doctor Names and Specializations

```
SELECT DoctorName, Specialization FROM Doctor;
```

Output:

DoctorName	Specialization
Dr. Mehta	Cardiologist
Dr. Patil	General
Dr. Sharma	Neurologist
Dr. Joshi	Orthopedic
Dr. Singh	Dermatologist

Q2. Retrieve Patients with Age Greater Than 40

```
SELECT * FROM Patient WHERE Age > 40;
```

Output:

PatientID	PatientName	Gender	Age	Disease
1	Aarav Joshi	Male	45	Diabetes
3	Ravi Patil	Male	52	BP
5	Kiran Shah	Male	60	Heart

Q3. Sort Doctors by Salary

```
SELECT * FROM Doctor ORDER BY Salary DESC;
```

Output:

DoctorId	DoctorName	Specialization	Salary	Exp
3	Dr. Sharma	Neurologist	90000.00	15
1	Dr. Mehta	Cardiologist	80000.00	10
4	Dr. Joshi	Orthopedic	70000.00	8
5	Dr. Singh	Dermatologist	65000.00	7
2	Dr. Patil	General	55000.00	5

Q4. Count Total Patients

```
SELECT COUNT(*) AS TotalPatients FROM Patient;
```

Output:

TotalPatients
5

Q5. Calculate Total Due Amount

```
SELECT SUM(DueAmount) AS TotalDue FROM Billing;
```

Output:

TotalDue
20000.00

PS 12 — Queries (Joins + View)

Q1. Display Patient with Doctor Details

```
SELECT p.PatientName, d.DoctorName, d.Specialization, a.AppintmentDate
FROM Patient p
JOIN Appointment a ON p.PatientID = a.PateintID
JOIN Doctor d ON a.DoctorId = d.DoctorId;
```

Output:

PatientName	DoctorName	Specialization	AppintmentDate
Aarav Joshi	Dr. Mehta	Cardiologist	2024-07-01
Priya Sharma	Dr. Patil	General	2024-07-02
Ravi Patil	Dr. Sharma	Neurologist	2024-07-03
Sneha More	Dr. Joshi	Orthopedic	2024-07-04
Kiran Shah	Dr. Singh	Dermatologist	2024-07-05

Q2. Display Appointment Details

```
SELECT a.AppointmnetID, p.PatientName, d.DoctorName, a.AppintmentDate, a.RoomNo
FROM Appointment a
JOIN Patient p ON a.PateintID = p.PatientID
JOIN Doctor d ON a.DoctorId = d.DoctorId;
```

Output:

AppointmnetID	PatientName	DoctorName	AppintmentDate	RoomNo
1	Aarav Joshi	Dr. Mehta	2024-07-01	101
2	Priya Sharma	Dr. Patil	2024-07-02	102
3	Ravi Patil	Dr. Sharma	2024-07-03	103
4	Sneha More	Dr. Joshi	2024-07-04	104
5	Kiran Shah	Dr. Singh	2024-07-05	105

Q3. Display Patient Billing Information

```
SELECT p.PatientName, b.TotalAmount, b.PaidAmount, b.DueAmount
FROM Patient p
JOIN Billing b ON p.PatientID = b.PatientID;
```

Output:

PatientName	TotalAmount	PaidAmount	DueAmount
Aarav Joshi	15000.00	15000.00	0.00
Priya Sharma	5000.00	3000.00	2000.00
Ravi Patil	20000.00	20000.00	0.00
Sneha More	8000.00	5000.00	3000.00
Kiran Shah	25000.00	10000.00	15000.00

Q4. Doctor with Highest Salary

```
SELECT DoctorName, Salary FROM Doctor
ORDER BY Salary DESC LIMIT 1;
```

Output:

DoctorName	Salary
Dr. Sharma	90000.00

Q5. Create Appointment View

```
CREATE VIEW AppointmentView AS
SELECT a.AppointmnetID, p.PatientName, d.DoctorName, a.AppintmentDate
FROM Appointment a
JOIN Patient p ON a.PateintID = p.PatientID
JOIN Doctor d ON a.DoctorId = d.DoctorId;

SELECT * FROM AppointmentView;
```

Output:

AppointmnetID	PatientName	DoctorName	AppintmentDate
1	Aarav Joshi	Dr. Mehta	2024-07-01
2	Priya Sharma	Dr. Patil	2024-07-02
3	Ravi Patil	Dr. Sharma	2024-07-03
4	Sneha More	Dr. Joshi	2024-07-04
5	Kiran Shah	Dr. Singh	2024-07-05

PS 32 & 36 — Sales Management: Stored Procedure & Function

Step 1: Create Table & Insert Data

```
CREATE TABLE Sales (  
    Sale_Id INT PRIMARY KEY AUTO_INCREMENT,  
    Product_Name VARCHAR(50),  
    Quantity INT,  
    Price DECIMAL(10,2)  
);  
  
INSERT INTO Sales (Product_Name, Quantity, Price) VALUES  
('Laptop',2,45000), ('Phone',5,15000), ('Tablet',3,20000),  
('Earbuds',10,3000), ('Monitor',1,12000);
```

Stored Procedure — Add_Sale

```
DELIMITER $$  
CREATE PROCEDURE Add_Sale(  
    IN p_name VARCHAR(50),  
    IN p_qty INT,  
    IN p_price DECIMAL(10,2)  
)  
BEGIN  
    INSERT INTO Sales (Product Name, Quantity, Price)  
    VALUES (p_name, p_qty, p_price);  
    SELECT 'Record Successfully Inserted' AS Message;  
END$$  
DELIMITER ;  
  
-- Call the procedure:  
CALL Add_Sale('Keyboard', 4, 2000);
```

Output:

Message
Record Successfully Inserted

Function — Total_Sales

```
DELIMITER $$  
CREATE FUNCTION Total_Sales()  
RETURNS DECIMAL(12,2)  
DETERMINISTIC  
BEGIN  
    DECLARE total DECIMAL(12,2);  
    SELECT SUM(Quantity * Price) INTO total FROM Sales;  
    RETURN total;  
END$$  
DELIMITER ;  
  
-- Use the function:  
SELECT Total_Sales() AS TotalSales;
```

Output:

TotalSales
281000.00

PS 33 & 37 — Employee Management: Triggers & Audit Log

Step 1: Create Tables & Insert Data

```
CREATE TABLE Employees (  
    EmployeeID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Department VARCHAR(50),  
    Salary DECIMAL(10,2)  
);  
  
CREATE TABLE Employee_Audit (  
    AuditID INT PRIMARY KEY AUTO_INCREMENT,  
    EmployeeID INT,  
    Action VARCHAR(10),  
    OldSalary DECIMAL(10,2),  
    NewSalary DECIMAL(10,2),  
    ActionDate DATETIME  
);  
  
INSERT INTO Employees VALUES  
(1,'Amit','IT',55000), (2,'Priya','Finance',62000),  
(3,'Ravi','HR',48000), (4,'Sneha','AI&DS',71000), (5,'Kiran','Marketing',40000);
```

Trigger for INSERT

```
DELIMITER $$  
CREATE TRIGGER after_emp_insert  
AFTER INSERT ON Employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO Employee_Audit (EmployeeID, Action, OldSalary, NewSalary, ActionDate)  
    VALUES (NEW.EmployeeID, 'INSERT', NULL, NEW.Salary, NOW());  
END$$  
DELIMITER ;
```

Trigger for UPDATE

```
DELIMITER $$  
CREATE TRIGGER after_emp_update  
AFTER UPDATE ON Employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO Employee_Audit (EmployeeID, Action, OldSalary, NewSalary, ActionDate)  
    VALUES (NEW.EmployeeID, 'UPDATE', OLD.Salary, NEW.Salary, NOW());  
END$$  
DELIMITER ;
```

Trigger for DELETE

```
DELIMITER $$  
CREATE TRIGGER after_emp_delete  
AFTER DELETE ON Employees  
FOR EACH ROW  
BEGIN  
    INSERT INTO Employee_Audit (EmployeeID, Action, OldSalary, NewSalary, ActionDate)  
    VALUES (OLD.EmployeeID, 'DELETE', OLD.Salary, NULL, NOW());  
END$$  
DELIMITER ;
```

Test Triggers & Check Audit Log

```
-- Test INSERT trigger
INSERT INTO Employees VALUES (6, 'Rahul', 'CS', 50000);

-- Test UPDATE trigger
UPDATE Employees SET Salary=60000 WHERE EmployeeID=1;

-- Test DELETE trigger
DELETE FROM Employees WHERE EmployeeID=6;

-- View audit log
SELECT * FROM Employee_Audit;
```

Output — Employee_Audit Table:

AuditID	EmployeeID	Action	OldSalary	NewSalary	ActionDate
1	6	INSERT	NULL	50000.00	2024-07-10 10:00:01
2	1	UPDATE	55000.00	60000.00	2024-07-10 10:00:02
3	6	DELETE	50000.00	NULL	2024-07-10 10:00:03

PS 34 & 35 — MongoDB CRUD, Aggregation & Indexing

Note: Run these commands in MongoDB Shell (mongosh) or MongoDB Compass.

PS 34 — CRUD Operations

Setup — Create Database & Insert Data

```
use userDB

db.users.insertMany([
  {name:"Aarav",age:20,city:"Pune",email:"aarav@mail.com"},
  {name:"Priya",age:22,city:"Mumbai",email:"priya@mail.com"},
  {name:"Ravi",age:19,city:"Nashik",email:"ravi@mail.com"},
  {name:"Sneha",age:21,city:"Pune",email:"sneha@mail.com"},
  {name:"Kiran",age:23,city:"Mumbai",email:"kiran@mail.com"}
])
```

READ — Find All Users

```
db.users.find()
```

Output:

name	age	city	email
Aarav	20	Pune	aarav@mail.com
Priya	22	Mumbai	priya@mail.com
Ravi	19	Nashik	ravi@mail.com
Sneha	21	Pune	sneha@mail.com
Kiran	23	Mumbai	kiran@mail.com

READ — Filter by City

```
db.users.find({city:"Pune"})
```

Output:

name	age	city
Aarav	20	Pune
Sneha	21	Pune

UPDATE — Change Age of Aarav

```
db.users.updateOne(
  {name:"Aarav"},
  {$set:{age:21}}
)
```

Output:

acknowledged	matchedCount	modifiedCount
true	1	1

DELETE — Remove Kiran

```
db.users.deleteOne({name:"Kiran"})
```

Output:

acknowledged	deletedCount
true	1

PS 35 — Aggregation & Indexing

Average Age of All Users

```
db.users.aggregate([
  { $group: { _id: null, avgAge: { $avg: '$age' } } }
])
```

Output:

_id	avgAge
null	21

Count Users by City

```
db.users.aggregate([
  { $group: { _id: '$city', count: { $sum: 1 } } }
])
```

Output:

_id	count
Pune	2
Mumbai	2
Nashik	1

Total Sales (Sales Collection)

```
use salesDB
db.sales.insertMany([
  {product:"Laptop",qty:2,price:45000},
  {product:"Phone",qty:5,price:15000},
  {product:"Tablet",qty:3,price:20000}
])

db.sales.aggregate([
  { $group: { _id:null, grandTotal:{ $sum:{ $multiply:['$qty','$price'] } } } }
])
```

Output:

_id	grandTotal
null	225000

Create Index & Check Performance

```
-- Create index on name field
db.users.createIndex({name: 1})
```

```
-- Output: { name: 1 }_1
```

```
-- Check all indexes  
db.users.getIndexes()
```

Output:

v	key	name
2	{_id:1}	_id_
2	{name:1}	{name:1}_1